

Unit: `HttpServletRequest` and `HttpServletResponse`

1. Introduction

In a Java Servlet application, communication takes place between the **client (browser)** and the **server**.

For example:

1. The user enters their name and email in an HTML form.
2. The browser sends this data to the server.
3. The Servlet receives and processes the data.
4. The Servlet sends a response back to the browser.

Java provides two important interfaces for this communication:

- `HttpServletRequest` → receives information **from the client**
- `HttpServletResponse` → sends information **to the client**

Remember:

Request = Data coming **to the server**

Response = Data going **back to the browser**

2. `HttpServletRequest`

Definition

`HttpServletRequest`: An interface that provides information about the HTTP request sent by the client/browser to the Servlet.

Purpose

It is mainly used when the Servlet needs to **receive or read data** sent by the user.

For example, it can be used to:

- Read HTML form inputs
- Get request parameters
- Access session-related information
- Access cookies
- Obtain other information associated with the incoming request

Key Method: `getParameter()`

The most commonly used method for receiving form data is:

```
request.getParameter("fieldName");
```

It retrieves the value associated with a particular form field.

Example

Suppose the HTML form contains:

```
<input type="text" name="name1">
```

The Servlet can retrieve the entered value using:

```
String myName = request.getParameter("name1");
```

Here:

- `request` → `HttpServletRequest` object
- `"name1"` → value of the HTML `name` attribute
- `myName` → stores the value entered by the user

Important Point

`getParameter()` **returns a String**.

Therefore, when receiving normal form data as text, you do not need to typecast the returned value.

Important for Exam:

```
String myName = request.getParameter("name1");
```

The parameter name must match the HTML input's **name attribute**, not its `id`.

3. `HttpServletResponse`

Definition

`HttpServletResponse`: An interface that provides methods for sending the Servlet's response back to the client/browser.

Purpose

After receiving and processing the user's data, the Servlet needs to send something back to the browser.

`HttpServletResponse` is used for this purpose.

For example, the Servlet can send:

- Text
- HTML output
- Response headers
- Other HTTP response information

4. `getWriter()` Method

To send text output to the browser, we commonly obtain a `PrintWriter` from the response object.

Syntax

```
PrintWriter out = response.getWriter();
```

Then we can use:

```
out.print("Hello");
```

to send output to the browser.

Example

```
PrintWriter out = response.getWriter();
```

```
out.print("Welcome " + myName);
```

If `myName` contains `Rahul`, the browser can display:

```
Welcome Rahul
```

5. `System.out.println()` VS `out.print()`

This is an important concept for beginners.

`System.out.println()`

```
System.out.println("Hello");
```

This prints the message to the **server-side console**, such as the Eclipse console.

It does **not** directly display the message on the user's browser.

`out.print()`

```
out.print("Hello");
```

This writes the response that is sent to the **client/browser**.

Difference

Feature	<code>System.out.println()</code>	<code>out.print()</code>
Output location	Server console	Browser/client
Object used	<code>System.out</code>	<code>PrintWriter</code>
Purpose	Debugging/server-side output	Sending response to browser
Example	<code>System.out.println("Hi");</code>	<code>out.print("Hi");</code>

Remember:

Console output → `System.out.println()`

Browser output → `out.print()`

6. Practical Example: HTML Form + Servlet

Suppose we want to create a simple form that accepts a user's **name** and **email**.

Step 1: Create the HTML Form

Example `index.html`:

```
<html>

<body>

<form action="submitForm">

    Name:

    <input type="text" name="name1">

    <br><br>

    Email:

    <input type="text" name="email1">
```

```
<br><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

Important Parts

```
name="name1"
```

and

```
name="email1"
```

are important because the Servlet uses these names to retrieve the values.

The form also contains:

```
action="submitForm"
```

This specifies the URL to which the form data is submitted.

Step 2: Create the Servlet

The Servlet can be created by extending `HttpServlet`.

```
@WebServlet("/submitForm")
```

```
public class MyServlet extends HttpServlet {
```

```
}
```

@WebServlet

@WebServlet("/submitForm") maps the Servlet to the URL `/submitForm`.

Therefore, when the form submits its data to:

```
submitForm
```

the server knows that **MyServlet** should handle the request.

Step 3: Receive the Form Data

Inside the Servlet, use the `HttpServletRequest` object.

```
String name = request.getParameter("name1");  
String email = request.getParameter("email1");
```

The Servlet now has access to the values entered by the user.

For example:

Name entered → Amit

Email entered → amit@gmail.com

Then:

```
name  = "Amit"  
email = "amit@gmail.com"
```

Step 4: Send the Response

Obtain a `PrintWriter` from `HttpServletResponse`.

```
PrintWriter out = response.getWriter();
```

Then send the output:

```
out.print("Welcome " + name);  
out.print("<br>");  
out.print("Your email is " + email);
```

The browser can display:

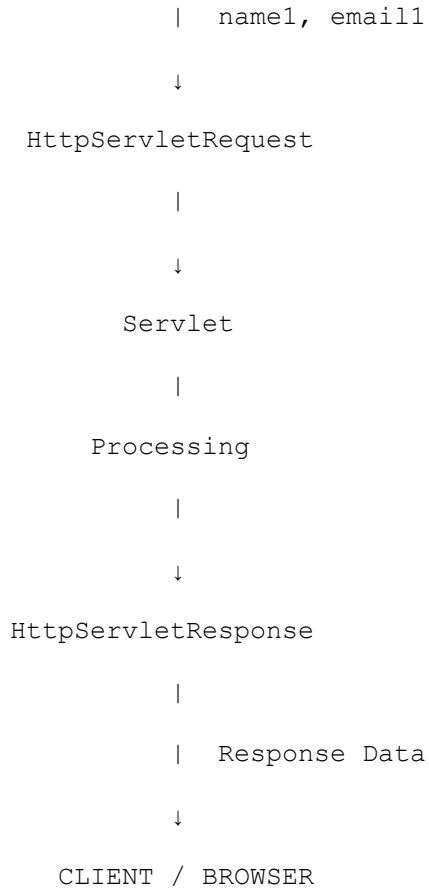
Welcome Amit

Your email is amit@gmail.com

7. Complete Basic Flow

The complete communication can be understood as:

```
CLIENT / BROWSER  
  
|  
  
|  Form Data
```



In simple words

1. **Browser** sends data.
2. **HttpServletRequest** receives that data.
3. **Servlet** processes the data.
4. **HttpServletResponse** sends the result.
5. **Browser** displays the result.

8. Request and Response Together

The two interfaces work as a pair.

```
String name = request.getParameter("name1");
```

```
PrintWriter out = response.getWriter();
```

```
out.print("Welcome " + name);
```

Here:

- `request` → receives data
- `getParameter()` → extracts form data
- `name` → stores the received value
- `response` → prepares the response
- `getWriter()` → obtains a writer
- `out.print()` → sends output to the browser

9. Important Methods

`HttpServletRequest`

`getParameter()`

Used to retrieve a parameter sent by the client.

```
String value = request.getParameter("name1");
```

`HttpServletResponse`

`getWriter()`

Used to obtain a `PrintWriter` for writing character data to the response.

```
PrintWriter out = response.getWriter();
```

Then:

```
out.print("Hello");
```

can be used to send output.

10. Interfaces, Not Classes

A very important examination point is that:

- **`HttpServletRequest` is an interface**
- **`HttpServletResponse` is an interface**

They are not ordinary concrete classes that you manually create using `new`.

The Servlet container/server provides the appropriate request and response objects when a request reaches the Servlet.

Conceptually:

Browser



Web Server / Servlet Container



Servlet



request + response objects

The Servlet uses these objects to communicate with the client.

11. `service()` Method

When a Servlet receives a request, the Servlet container invokes its processing mechanism and provides the request and response objects.

A basic form is:

```
public void service(HttpServletRequest request,
                    HttpServletResponse response)
```

The two parameters represent:

- **`HttpServletRequest request`** → incoming request
- **`HttpServletResponse response`** → outgoing response

The Servlet can therefore use:

```
request.getParameter(...)
```

to receive data and:

```
response.getWriter()
```

to send data.

12. Complete Conceptual Example

```
@WebServlet("/submitForm")
```

```
public class MyServlet extends HttpServlet {
```

```

    public void service(HttpServletRequest request,
                        HttpServletResponse response)
        throws IOException {

        String name = request.getParameter("name1");
        String email = request.getParameter("email1");

        PrintWriter out = response.getWriter();

        out.print("Welcome " + name);
        out.print("<br>");
        out.print("Email: " + email);
    }
}

```

Working of this code

1. The browser submits the HTML form.
2. The request reaches /submitForm.
3. The Servlet container calls the Servlet.
4. request.getParameter("name1") retrieves the name.
5. request.getParameter("email1") retrieves the email.
6. response.getWriter() creates/returns a PrintWriter.
7. out.print() sends the result to the browser.

13. Commonly Confused Points

Request vs Response

Request means information travelling:

Client → Server

Response means information travelling:

Server → Client

System.out VS PrintWriter

`System.out` is generally used for output on the **server console**.

`PrintWriter` obtained from `response.getWriter()` is used to write the **HTTP response** sent to the client.

name VS id

For:

```
<input type="text" name="name1">
```

the Servlet retrieves the value using:

```
request.getParameter("name1");
```

The value passed to `getParameter()` should correspond to the form field's **name**.

14. Quick Comparison

Feature	<code>HttpServletRequest</code>	<code>HttpServletResponse</code>
Direction	Client → Server	Server → Client
Main purpose	Receive/read data	Send/write response
Used by	Servlet	Servlet
Common method	<code>getParameter()</code>	<code>getWriter()</code>
Example	Read user's name	Display user's name
Interface?	Yes	Yes

15. Important Exam Points

Important for Exam:

- `HttpServletRequest` is used to access information from the incoming client request.
- `HttpServletResponse` is used to create/send the response to the client.
- `request.getParameter()` is commonly used to retrieve HTML form data.
- `getParameter()` returns a **String**.
- `response.getWriter()` returns a **PrintWriter**.
- `out.print()` can send text as part of the response to the browser.
- `System.out.println()` prints to the server-side console, not directly to the browser.
- Both `HttpServletRequest` and `HttpServletResponse` are interfaces.
- The Servlet container provides the request and response objects.

16. Key Points

- **Request = receiving data**
- **Response = sending data**
- **HttpServletRequest** handles information coming from the client.
- **HttpServletResponse** handles information going back to the client.
- **getParameter()** retrieves form values.
- The parameter name normally matches the HTML **name attribute**.
- **getParameter()** returns **String**.
- **getWriter()** provides a **PrintWriter**.
- **out.print()** sends output to the browser.
- **System.out.println()** sends output to the server console.
- Both request and response are **interfaces**.

17. Summary

HttpServletRequest and **HttpServletResponse** are two fundamental interfaces used in Servlet-based web applications.

HttpServletRequest is responsible for receiving information from the client. For example, `request.getParameter("name1")` can retrieve a value submitted through an HTML form.

HttpServletResponse is responsible for sending information back to the client. Using `response.getWriter()`, the Servlet obtains a **PrintWriter**, and `out.print()` can be used to send text to the browser.

The complete idea can be remembered as:

Client

|

| Request

↓

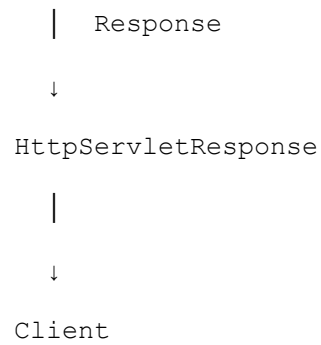
HttpServletRequest

|

↓

Servlet

|



One-line memory trick:

Request brings data IN → Servlet processes it → Response sends data OUT.

Video Details

Video title: #7 HttpServletRequest and HttpServletResponse Theory || Servlet & JSP Tutorials for Beginners

Video link: <https://youtu.be/dlG8WeoOlsg?si=cfMndiyip5w1fay4>